GPU-Accelerated Numerical Analysis of Blood Flow Through Arteries Using the Finite Volume Method (FVM)

Objective and Scope:

The primary objective of this report is to provide a thorough analysis of blood flow through arteries utilizing the Finite Volume Method. This investigation seeks to deepen our understanding of how various geometric and physiological factors impact crucial hemodynamic parameters. The report specifically focuses on key areas, including:

The sensitivity of coronary hemodynamic to the structural characteristics of blood vessels encompasses factors such as bifurcation angles, positional relationships, diameter ratios, and the tortuosity of arteries. The influence of stenotic lesions—narrowing in the arterial path—on flow patterns, pressure distributions, and wall shear stress is vital for understanding cardiovascular health. Implementing non-Newtonian fluid models is essential for accurately representing the complex rheological behaviour of blood, which is essential for realistic simulations.

The application of non-Newtonian fluid models to capture the complex rheological behaviour of blood. This report contributes to the growing body of knowledge on arterial hemodynamic and its implications for cardiovascular health and disease by synthesizing theoretical frameworks with computational simulations and experimental validations. The findings have potential applications in predicting disease progression, designing medical interventions, and developing personalized treatment strategies based on patient-specific vascular geometries.

Left Anterior Descending Artery (LAD) Specifications

The Left Anterior Descending (LAD) artery is the principal branch of the left coronary artery (LCA), responsible for supplying oxygenated blood to the anterior myocardium. Its clinical significance is underscored by its designation as the "widow-maker" artery due to the high mortality risk associated with proximal occlusions.

Segmental Classification

For precise hemodynamic analysis, the LAD is divided into four anatomical segments:

1. **Proximal LAD** (origin to first diagonal branch D1)
2. **Mid-Proximal LAD** (D1 to D2)
3. **Mid-Distal LAD** (D2 to D3)
4. **Distal LAD** (beyond D3 to cardiac apex)

Geometric and Hemodynamic Parameters

This study focuses on the proximal LAD segment with the following characteristics:

- **Density of blood**: 1060 kg/m^3
- **Diameter:** 3.5 mm (0.0035 m) [Range: 3.1-3.7 mm based on clinical data]
- **Length:** 14 cm (0.14 m) [Anatomically representative range: 10-15 cm]
- **Time-Averaged Mean Velocity:** 0.39 m/s (39 cm/s) [Physiological range: 0.15-0.40 m/s]

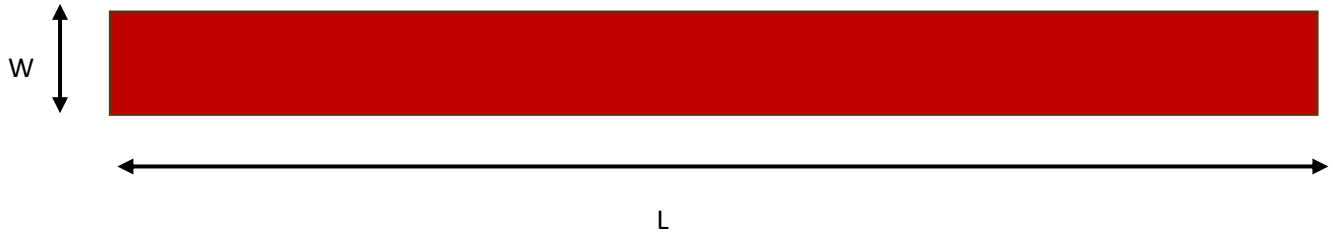
***Data reference**:* Ali, Sajid, et al. [2024], *[Journal of X-Ray Science and Technology]*

Numerical Study:

Numerical Study Conditions

1. **Flow Assumptions**
    - The flow is modelled as unsteady, incompressible, and laminar, as blood flow is a time-dependent physiological process.
    - The non-Newtonian, shear-thinning behaviour of blood is considered, where viscosity decreases with increasing shear rate.
2. **Computational Domain**
    - A 2D Cartesian grid is employed for the numerical simulations.
    - The flow direction is defined from left to right (positive *x*-direction, $Ue>0$).
3. **Rheological Model**
    - The Bird-Carreau model is used to characterize the shear-thinning behaviour of blood, with viscosity defined
4. **Boundary Conditions**
    - **Inlet:** inlet velocity profile as constant inflow.
    - **Outlet:** Pressure-based boundary condition (e.g., zero pressure gradient).
    - **Walls:** No-slip condition ($u=0u=0$).
5. **Numerical Scheme**
    - The Finite Volume Method (FVM) is applied for spatial discretization.
    - A first-order Euler explicit method is used to handle the unsteady flow characteristics.

Geometry of the artery:



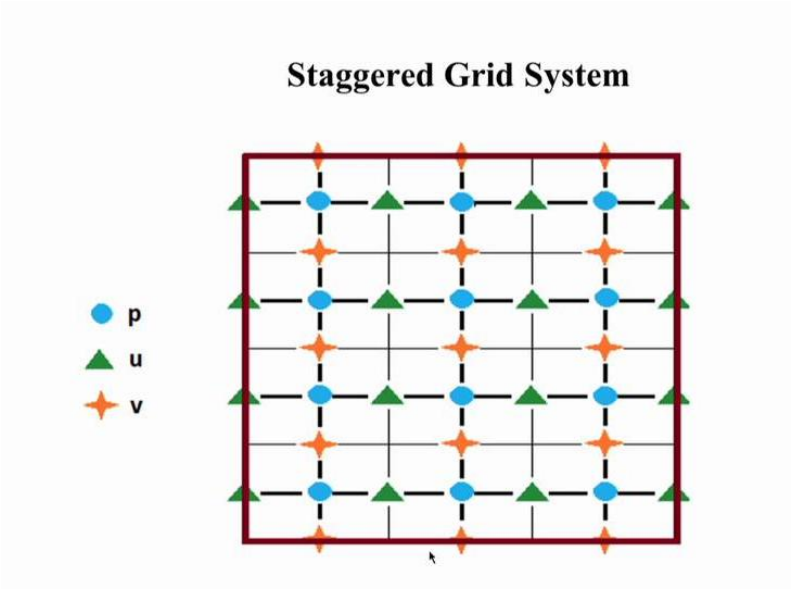
Domain Dimensions:

- Length = 0.14 m
- Width = 0.0035m
- Cell size in x directions = 0.0001 m
- Cell size in y direction = 0.0001 m

Staggered Grid Approach:

The **staggered grid** is used for spatial discretization method

- **Velocity components** (u, v) are stored at **cell faces**.
- **Pressure (p) and scalar quantities** (e.g., viscosity) are stored at cell center.



Spatial Discretization & Iterative Solution Methods

1. Convection Term: QUICK Scheme

- Scheme: Third-order QUICK (Quadratic Upstream Interpolation for Convective Kinematics)
- Formulation:
  For a generic variable ϕ (e.g., velocity component):

$$\phi f = \{\phi UP + 8\psi(3\phi D - 2\phi UP - \phi UU)$$

2. Diffusion Term: Central Difference Scheme

- Scheme: Second-order Central Difference

3.Validation & Stability:

- **Grid Sensitivity**: Verify QUICK accuracy via grid refinement studies.
- **CFL Condition**: For explicit convection, CFL=uΔt/Δx≤ 1
- **Shear-Thinning**: Ensure η(γ˙)$\eta(\gamma\dot{})$ is updated iteratively within GS loops.

Governing equations

unsteady Incompressible flow.

① conservation of mass.

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0.$$

② conservation of momentum equation:-

① x momentum equation:-

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -\frac{1}{\rho}\frac{\partial P}{\partial x} + \frac{1}{\rho}\left[\frac{\partial}{\partial x}\left(\mu\frac{\partial u}{\partial x}\right) + \frac{\partial}{\partial y}\left(\mu\frac{\partial}{\partial y}\right)\right]$$

② y momentum equation:-

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = -\frac{1}{\rho}\frac{\partial P}{\partial y} + \frac{1}{\rho}\left[\frac{\partial}{\partial x}\mu\left(\frac{\partial v}{\partial x}\right) + \frac{\partial}{\partial y}\left(\mu\frac{\partial v}{\partial y}\right)\right]$$

③ μ → non-newtonian (shear thinning fluid flow).

The Bird - carreau method implemented,

$$\mu = \mu_\infty + (\mu_0 - \mu_\infty)\left[1 + \lambda(\dot{\gamma})^2\right]^{\frac{n-1}{2}}.$$

$\mu_\infty \rightarrow 0.0035$ Pa·s

$\mu_0 \rightarrow 0.056$ Pa·s

$\lambda \rightarrow 3.313$ s

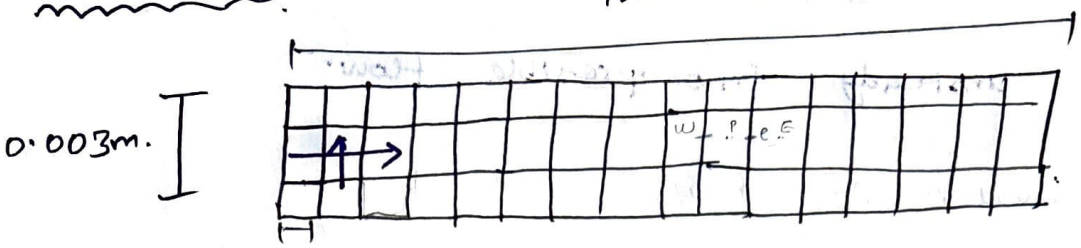
$n = 0.6$

$$\dot{\gamma} = \sqrt{\frac{1}{2}\Sigma_i \Sigma_j d_{ij} d_{ji}} \rightarrow \text{shear rate.}$$

$$\dot{\gamma} = \left[2\left[\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2\right] + \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right)^2\right]^{\frac{1}{2}}$$

Discretization:-

12cm = 0.12m.



$\Delta y = 1\text{mm} = 0.001\text{m}$

$\Delta x = 0.001\text{m}$.

$NX = 120$

$NY = 3$

$u_e > 0$.

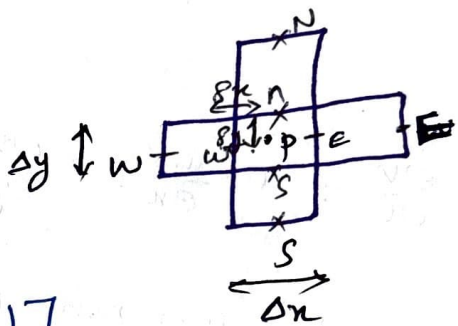
$v_n > 0$

Spatial discritization:-

Advection diffusion scheme:

general form.

$$\frac{\partial u u}{\partial x} = \frac{d}{dx}\, u_e \left(\frac{\partial u}{\partial x}\right).$$



① diffusion:- (x momentum)

$$\frac{\partial}{\partial x}\left(\mu \left(\frac{\partial u}{\partial x}\right) + \frac{\partial}{\partial y}\, \mu \left(\frac{\partial u}{\partial y}\right)\right]$$

$\phi = u$.

For 2D equation.

general form, $\frac{d}{dx}\left(\Gamma \frac{d\phi}{dx}\right)$.

$A_e = \Delta x$

$A_w = \Delta x$

$A_n =$

$$\left[\mu_e A_e \left(\frac{\partial u}{\partial x}\right)\right] = \frac{\mu_e A_e}{} \left[\frac{\phi_E - \phi_P}{(\delta x)}\right]$$

$$\left[\mu_w A_w \left(\frac{\partial u}{\partial x}\right)\right] = \mu_w A_w \frac{[\phi_P - \phi_w]}{\delta x}.$$



$$\left[\mu_n A_n \frac{\partial u}{\partial y}\right] = \mu_n A_n \frac{[\phi_N - \phi_P]}{\delta y}.$$

$$\left[\mu_s A_s \frac{\partial u}{\partial y}\right] = \mu_s A_s \frac{[\phi_P - \phi_s]}{\delta y}.$$

2D diffusion terms-

$$\left[\frac{\mu_e A_e}{\delta x}[\phi_E - \phi_P] - \frac{\mu_w A_w}{\delta x}[\phi_P - \phi_W]\right]$$

$$+\left[\frac{\mu_n A_n}{\delta y}[\phi_N - \phi_P] - \frac{\mu_s A_s}{\delta y}[\phi_P - \phi_S]\right]$$

general form derivatives. to x momentum,

$$\frac{\mu_e A_e}{\delta x}(u_E - u_P] = D_e[u_E - u_P]. \quad \left[D_e = \frac{A_e \mu_e}{\delta x}\right]$$

like for others also,

$$D_e[\phi_E - \phi_P] - D_w[\phi_P - \phi_W) + D_n(\phi_N - \phi_P) + D_s(\phi_P - \phi_S).$$

x momentum the Diffusion term,

$$\frac{1}{\delta}\left[D_e(u_E - u_P] - D_w(\phi_P - u_W) + D_n[\phi_N - u_P] + D_s[u_P - u_S]\right]$$

Y momentum equation.

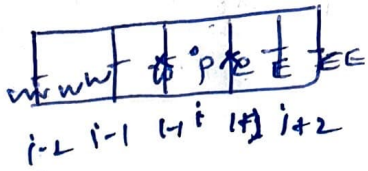
$$\frac{1}{\delta}\left[D_e[v_E - v_P] - D_w[v_P - v_W] + D_n[v_n - v_P] + D_s[v_P - v_S].\right]$$

Quick scheme For 1D momentum advection

Terms:

x momentum equation:-

$$\frac{\partial(uu)}{\partial x} + \frac{\partial(vu)}{\partial y}$$



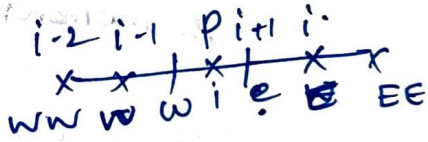
① East face:- $\frac{\partial uu}{\partial x}$ (direction of x)

General form of Quick scheme.

$$\phi_{face} = \frac{3}{8}\phi_{upwind} + \frac{6}{8}\phi_{central} - \frac{1}{8}\phi_{downwind}.$$

$$\phi_e = \frac{3}{8}\phi_E + \frac{6}{8}\phi_P - \frac{1}{8}\phi_W$$

$$\phi_w = \frac{3}{8}\phi_E + \frac{6}{8}\phi_P -$$



(or)

General form of Quick.

$$\phi_{face} = \frac{6}{8}\phi_{i+1} + \frac{3}{8}\phi_i - \frac{1}{8}\phi_{(i-1)}.$$

$$\phi_e = \frac{6}{8}\phi_P + \frac{3}{8}\phi_E - \frac{1}{8}\phi_W.$$

$$\phi_w = \frac{6}{8}\phi_W + \frac{3}{8}\phi_P - \frac{1}{8}\phi_{WW}.$$

General Form of Quick Scheme.

$$\phi_{face} = \frac{3}{8}\phi_{upwind} + \frac{6}{8}\phi_{central} - \frac{1}{8}\phi_{downwind}.$$

mathematically,

$$\phi_e = \frac{3}{8}\phi_E + \frac{6}{8}\phi_P - \frac{1}{8}\phi_W$$

$$\phi_w = \frac{3}{8}\phi_P + \frac{6}{8}\phi_W - \frac{1}{8}\phi_{WW}.$$

Substituting the u velocity.

$$u_e = \frac{3}{8}\phi_E + \frac{6}{8}\phi_P - \frac{1}{8}\phi_W.$$

$$u_w = \frac{3}{8}\phi_P + \frac{6}{8}u_W - \frac{1}{8}u_{WW}.$$

$$u_n = \frac{3}{8}u_N + \frac{6}{8}u_P - \frac{1}{8}u_S$$

$$u_s = \frac{3}{8}u_P + \frac{6}{8}u_S - \frac{1}{8}u_{SS}.$$

Substitute this equation in the x momentum equation.

$$F_e\left[\frac{3}{8}\phi_E + \frac{6}{8}\phi_P - \frac{1}{8}\phi_W\right] - F_w\left[\frac{3}{8}\phi_P + \frac{6}{8}u_W - \frac{1}{8}u_{WW}\right]$$

$$+ F_n\left[\frac{3}{8}\phi_N + \frac{6}{8}\phi_P - \frac{1}{8}u_S\right] + F_s\left[\frac{3}{8}u_P + \frac{6}{8}u_S - \frac{1}{8}u_{SS}\right].$$

$$= \left[\frac{3}{8}F_e\phi_E + \frac{6}{8}F_e\phi_P - \frac{1}{8}F_e\phi_W\right] - F_w\left[\frac{3}{8}\phi_P\right]$$

$$Fe\left[\tfrac{3}{8}\phi_E + \tfrac{6}{8}\phi_P - \tfrac{1}{8}\phi_W\right] - Fw\left[\tfrac{3}{8}\phi_P + \tfrac{6}{8}\phi_W - \tfrac{1}{8}\phi_{WW}\right]$$

$$+ Fn\left[\tfrac{3}{8}\phi_N + \tfrac{6}{8}\phi_P - \tfrac{1}{8}\phi_S\right] - Fs\left[\tfrac{3}{8}\phi_P + \tfrac{6}{8}\phi_S - \tfrac{1}{8}\phi_{SS}\right]$$

Solving the above equation.

$$\left[\tfrac{3}{8}Fe\phi_E + \tfrac{6Fe}{8}\phi_P - \tfrac{1}{8}\phi_W Fe\right] - \tfrac{3Fw}{8}\phi_P - \tfrac{6Fw}{8}\phi_W + \tfrac{Fw}{8}\phi_{WW}$$

$$+\tfrac{3Fn}{8}\phi_N + \tfrac{6Fn}{8}\phi_P - \tfrac{Fn}{8}\phi_S - \tfrac{3Fs}{8}\phi_P + \tfrac{6Fs\phi_S}{8} + \tfrac{1}{8}Fs\phi_{SS}\Big]$$

$$\tfrac{3}{8} = 0.375\,, \quad \tfrac{6}{8} = 0.75, \quad \tfrac{1}{8} = 0.125$$

Substituting the above equation.

$$[0.375]\,Fe\phi_E + [0.75]\,Fe\phi_P - [0.125\,Fe\phi_W]$$

$$- [0.375\,\phi_P Fw] - (0.75)\,Fw\phi_W + (0.125)\,Fw\phi_{WW}$$

$$+ (0.375]\,Fn\phi_N + (0.75\,Fn\phi_P] - (0.125)\,Fn\phi_S$$

$$- 0.375\,[Fs\phi_P] - 0.75[Fs\phi_S] + (0.125]\,Fs\phi_{SS}$$

$$[0.375\,Fe\phi_E] - [0.125\,Fe\phi_W] - 0.75(Fw\phi_W) + [0.125\,Fw\phi_{WW}]$$

$$+0.375\,Fn\phi_N - 0.125\,Fn\phi_S - 0.75\,Fs\phi_S + 0.125\,Fs\phi_{SS}$$

$$+ (0.75\,Fe\phi_P - 0.375\,\phi_P Fw) + [0.75\,Fn\phi_P]$$

$$[-0.375\,Fs\phi_P]$$

$$3/8\ Fe\ \phi_E - 1/8\ Fe\ \phi_W - 6/8\ Fw\ \phi_W + 1/8\ Fw\ \phi_{WW}$$

$$+ 3/8\ Fn\ \phi_N - 1/8\ Fn\ \phi_S - 6/8\ Fs\ \phi_S + 1/8\ Fs\ \phi_{SS}$$

$$+ \left[6/8\ Fe - 3/8\ Fw + 6/8\ Fn - 3/8\ Fs \right] \phi_P$$

- correcting these equations to ϕ formation,

$$3/8\ Fe\ \phi_E - \left[1/8\ Fe + 6/8\ Fw \right] \phi_W + Fw/8\ \phi_{WW}$$

$$+3/8\ Fn\ \phi_N - \left[1/8\ Fn - 6/8\ Fs \right] \phi_S + 1/8\ Fs\ \phi_{SS}.$$

$$+ \left[6/8\ Fe - 3/8\ Fw + 6/8\ Fn - 3/8\ Fs \right] \phi_P$$

Now Substitute the 2D Diffusion term,

$$3/8\ Fe\ \phi_E - \left[1/8\ Fe + 6/8\ Fw \right] \phi_W + Fw/8\ \phi_{WW} + 3/8\ Fn\ \phi_N$$

$$+\left[1/8\ Fn - 6/8\ Fs \right] \phi_S + 1/8\ Fs\ \phi_{SS} + \left[6/8\ Fe - 3/8\ Fw + 6/8\ Fn - 3/8\ Fs \right] \rightarrow \phi_P$$

~~$1/8 \left[3\ Fe\ \phi_E \ldots Fw \right] \phi_W + Fw\ \phi_{WW} + 3\ Fn\ \phi_N$~~

$$= De\ \phi_E - De\ \phi_P - Dw\ \phi_P + Dw\ \phi_W + Dn\ \phi_N$$

$$- Dn\ \phi_P - \phi_P Ds + Ds\ \phi_S$$

$$\tfrac{3}{8} Fe \phi_E - [\tfrac{1}{8} Fe + \tfrac{6}{8} Fw] \phi_W + \tfrac{1}{8}[Fw \phi_{WW}]$$

$$+ \tfrac{3}{8} Fn \phi_N - [\tfrac{1}{8} Fn + \tfrac{6}{8} Fs] \phi_S + \tfrac{1}{8} Fs \phi_{SS}$$

$$+ [\tfrac{6}{8} Fe - \tfrac{3}{8} Fw + \tfrac{6}{8} Fn - \tfrac{3}{8} Fs] \phi_P$$

$$= De \phi_E - De \phi_P - Dw \phi_P + Dw \phi_W + Dn \phi_N$$

$$- Dn \phi_P - Ds \phi_P + Ds \phi_S$$

Rewriting the above equation in the ϕ name.

$$[\tfrac{3}{8} Fe - De] \phi_E - [\tfrac{1}{8} Fe + \tfrac{6}{8} Fw + Dw] \phi_W$$

$$+ \tfrac{3}{8} [Fn - Dn] \phi_N - [\tfrac{1}{8} Fn + \tfrac{6}{8} Fs + Ds] \phi_S$$

$$+ \tfrac{1}{8} Fw \phi_{WW} + \frac{Fs \phi_{SS}}{8}$$

$$= - [\tfrac{6}{8} Fe - \tfrac{3}{8} Fw + \tfrac{6}{8} Fn - \tfrac{3}{8} Fs] \phi_P$$

$$- De \phi_P - Dw \phi_P - Dn \phi_P - Ds \phi_S.$$

$$= - \tfrac{6}{8} Fe + \tfrac{3}{8} Fw - \tfrac{6}{8} Fn + \tfrac{3}{8} Fs$$

$$= [-\tfrac{6}{8} Fe - \phi_E] + [\tfrac{3}{8} Fe - Dw) - \tfrac{6}{8} Fn - Dw + \tfrac{3}{8} Fs - Ds]$$

ϕ_P.

Final Equation of advection-diffusion problem can be, simplified It,

$$a_e \phi_E = \left[\tfrac{3}{8} F_e - D_e\right] \phi_E$$

$$a_w \phi_W = -\left[\tfrac{1}{8} F_e + \tfrac{6}{8} F_w + D_w\right] \phi_W$$

$$a_n \phi_N = \left[\tfrac{3}{8} F_n - D_n\right] \phi_N$$

$$a_s \phi_S = -\left[\tfrac{1}{8} F_n + \tfrac{6}{8} F_s + D_s\right] \phi_S$$

$$a_{ww} \phi_{ww} = \tfrac{1}{8} F_w \phi_{ww}.$$

$$a_{ss} \phi_{ss} = \tfrac{1}{8} F_s \phi_{ss}.$$

$$a_p \phi_p = -\tfrac{6}{8} F_e - \phi_E + \left[\tfrac{3}{8} F_w - D_w\right] - \tfrac{6}{8} F_n - D_n + \tfrac{3}{8} F_s - D_s$$

$$a_p \phi_p = a_w \phi_W + a_e \phi_E + a_n \phi_N + a_s \phi_S + a_w \phi_{ww} + a_s \phi_{ss}$$

$$a_p \phi_p = \sum a_{nb} \phi_{nb}$$

Finite volume Formulation. For pressure.

The general Equation of FVM.

$$\nabla(\rho \vec{V} \phi) = S_\phi + \nabla(\tau_\phi \nabla \phi)$$

$$S_\phi = -\nabla p$$

we know that

$$a_p \phi_p = \sum_{nb} a_{nb} \phi_{nb}$$

$$\iiint_{CV} \nabla p \, dV = \iint_{CS} p \cdot dA = \frac{P_e - P_w}{\delta x}$$

$$= -\left[\left[\frac{P_E - P_P}{\delta}\right] - \left[\frac{P_P - P_W}{\delta}\right]\right] \qquad A_e = A_w = \frac{\Delta x}{\delta} = \frac{\Delta y}{2}$$

$$= -[P_E - P_W] A_e$$

In y.

$$a_p \phi_p = \sum_{nb} a_{nb} \phi_{nb} + [P_W - P_E] A_e.$$

$$a_p \phi_p = \sum_{nb} a_{nb} \phi_{nb} + [P_S - P_N] A_n.$$

Rewrite this as in velocity form,

$$a_p u_p = \sum_{nb} a_{nb} \phi_{nb} + [P_W - P_E] A_e.$$

$$a_p v_p = \sum_{nb} a_{nb} \phi_{nb} + [P_S - P_N] A_n.$$

The ℓt must satisfy

$$F_e - F_w + F_n - F_s = 0$$

$$\boxed{u_e A_e - u_w A_w + v_n A_n - v_s A_s = 0.}$$

~~Ubby.~~

~~[illegible]~~

SEMI Implicit method For pressure Linked Equation.

$$\frac{u^{n+1} - u^n}{\Delta t} + [\text{convection}]^n = -\frac{1}{\rho}\left[\frac{\partial p}{\partial n}\right]^{n+1} + \nu\,[\text{diffusion}]^n$$

$$\frac{u^* - u^n}{\Delta t} + [\text{convection}]^n = -\frac{1}{\rho}\left[\frac{\partial p}{\partial x}\right]^n + \nu\,[\text{diffusion}]^n$$

From the Finite volume method.

$$a_e u_e = \sum_{nb} a_{nb} u_{nb} + (P_w - P_e) A_e \quad \rightarrow (1)$$

$$a_e v_e = \sum_{anb} a_{nb} v_{nb} + [P_s - P_N] A_n \quad \rightarrow (2)$$

$$F_e - F_w + F_n - F_s = 0$$

$$\rho u_e A_e + \rho u_w A_w + \rho v_n A_n + \rho v_s A_s = 0 \quad \rightarrow (3)$$

prediction step:-

$$a_e u_e^* = \sum_{nb} a_{nb} u_{nb}^* + (P_w^* - P_e^*) A_e \quad \rightarrow (4)$$

$$a_e v_e^* = \sum_{nb} a_{nb} v_{nb}^* + [P_s^* - P_N^*] A_e. \quad \rightarrow (5)$$

① − ④

$$a_e (u_o - u_e^*) = \sum_{nb} a_{nb} (u_{nb} - u_{nb}^*) + [P_W - P_E] A_e$$

→ ⑥

$$a_n (v_n - v_n^*) = \sum_{nb} a_{nb} (v_{nb} - v_{nb}^*) + [P_S - P_N] A_n$$

$$- [P_{S0}^* - P_N^*] A_n]$$

→ ⑦

PRIME Algorithm.

1. Compute the stresses.
2. Compute the velocities
3. Compute the pressure and correct velocity.
4. Compute the stress using the new velocities.
5. Compute the pressure again. Correct the velocity.
6. Return to step 1 again until convergence.

Discritization of Equations:-

① Cell size = 1mm = 0.001m.

speed u = 0.39 m/s

time step = 0.0001

$$CFL = \frac{u \Delta t}{\Delta x} = \frac{0.39 \times 0.0001}{0.001}$$

$$\boxed{CFL = 0.039}$$

② Discritization of the shear rate calculation.

$$\dot{\gamma} = \left[2 \left[\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 \right] + \left[\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right]^2 \right]^{1/2}$$

$$\frac{\partial u}{\partial x} = \frac{u_{i+1j} - u_{ij}}{\delta x}$$

$$\frac{\partial v}{\partial y} = \frac{v_{ij+1} - v_{ij}}{\delta y}$$

$$\frac{\partial u}{\partial y} = \frac{u_{i+1} - u_{i-1}}{2\delta y}$$

$$\frac{\partial v}{\partial x} = \frac{v_{ij+1} - v_{ij-1}}{2\delta x}$$

$$\left[\frac{\partial u}{\partial y} + \frac{\partial v}{\partial y} \right] = \left[\frac{u_{i+1j} - u_{i-1j}}{2\delta y} + \frac{v_{ij+1} - v_{ij}}{2\delta x} \right]^2$$

$$\dot{\gamma} = \left[2 \left[\left[\frac{u_{i+1j} - u_{ij}}{\delta x} \right]^2 + \left[\frac{v_{ij+1} - v_{ij}}{\delta y} \right] \right]^2 \right]$$

Boundary conditions:-

* Inlet Velocity (u) = 0.39 m/s.

* wall boundary condition = No slip
  Velocity = 0

* outlet => Pressure outlet set to be zero.
  Neuman boundary condition.

mass conservation:-

$$\int_{VP} (\nabla u) dV = \sum_f u_f S_f = 0.$$

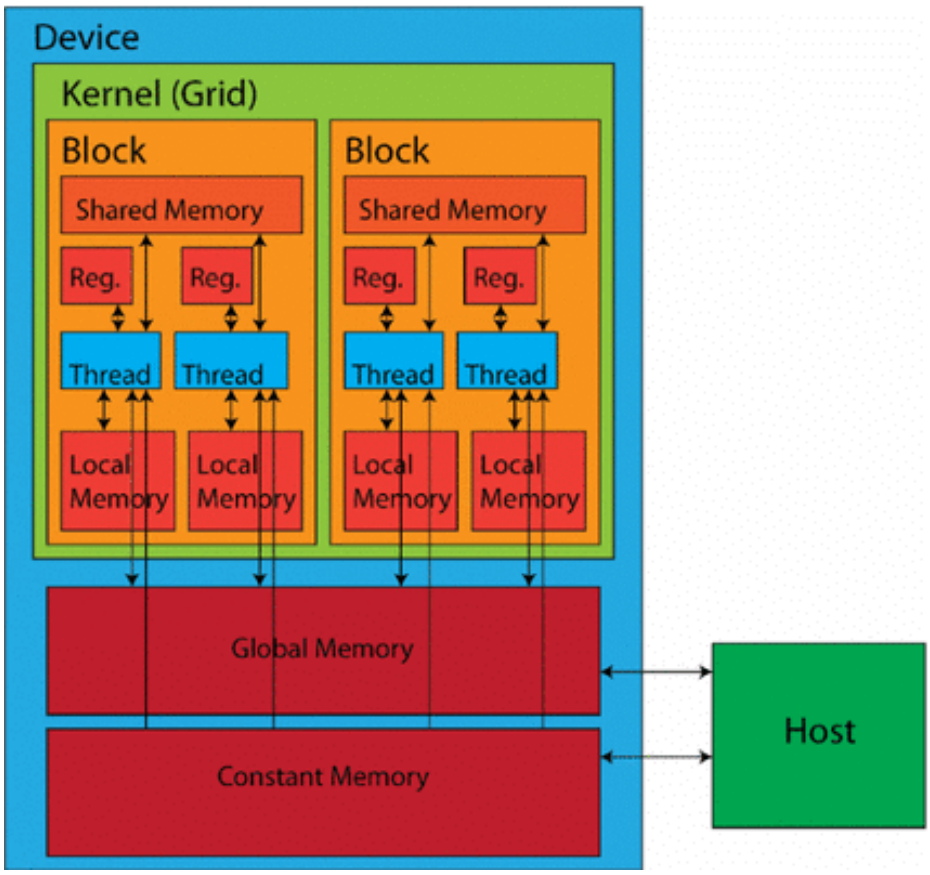
For the 2D cartisian,

$$(u_e A_e - u_w A_w) + (v_n A_n - v_s A_s) = 0.$$

GPU Architecture:

Introduction to GPU Architecture

Graphics Processing Units (GPUs) are specialized processors designed to accelerate graphics rendering and parallel computations. Modern GPUs have evolved into highly parallel, multithreaded, manycore processors with tremendous computational horsepower and very high memory bandwidth



Key Architectural Features

1. Parallel Processing Design

- **Massive parallelism**: GPUs contain thousands of smaller, more efficient cores (compared to CPUs)
- **SIMT architecture**: Single Instruction, Multiple Thread execution model
- **Deeply pipelined**: Designed to hide memory latency through massive multithreading

2. Core Components

- **Streaming Multiprocessors (SMs)**: Fundamental processing units containing:
    - CUDA cores (NVIDIA) or Stream processors (AMD)
    - Special function units (SFUs)
    - Tensor cores (in modern GPUs for AI acceleration)
- **Memory hierarchy**:
    - Global memory (high-capacity GDDR/HBM)
    - Shared memory/L1 cache
    - Registers (fastest, per-thread)
- **Interconnect**: High-speed bus between components

3. Memory Architecture

- **Hierarchical memory model** with different access speeds
- **Coalesced memory access** patterns for optimal performance
- **Memory controllers** optimized for high bandwidth

Comparison with CPU Architecture

Feature	CPU	GPU
Cores	Few (2-64)	Many (1000s)
Threads per core	1-2 (typically)	100s
Cache hierarchy	Large, complex	Smaller, simpler
Best for	Serial code	Parallel workloads
Clock speed	Higher	Lower

Blood Flow Simulation Algorithm

This algorithm simulates unsteady, incompressible, non-Newtonian blood flow through arteries using the Bird-Carreau model on a staggered grid with the SIMPLE algorithm implemented in CUDA.

Algorithm Steps

1. Initialization Phase
   - ✓ Define physical parameters for blood (density, viscosity parameters)
   - ✓ Set up computational domain (artery dimensions, grid resolution)
   - ✓ Allocate memory for velocity, pressure, viscosity, and intermediate variables
   - ✓ Initialize flow velocity profile at inlet
   - ✓ Set initial pressure gradient and viscosity distribution

2. Time Advancement Loop
   - ⊗ For each time step until reaching the desired physical simulation time:
   - ⊗ Store Previous Values - Save current velocity fields for residual calculation
3. SIMPLE Algorithm Iteration:

   Repeat until convergence or maximum iterations:

   Apply Boundary Conditions:

   - o Enforce parabolic velocity profile at inlet
   - o Set zero gradient at outlet for velocity
   - o Apply no-slip condition at walls
   - o Set pressure boundary conditions

   Momentum Prediction:

   - o Solve x-momentum equation using QUICK scheme for convection
   - o Solve y-momentum equation using QUICK scheme for convection
   - o Use central differencing for diffusion terms
   - o Calculate intermediate velocity fields (u* and v*)

Calculate Divergence

⇒ Compute velocity field divergence for pressure correction

Pressure Correction:

⇒ Solve pressure Poisson equation using Gauss-Seidel with SOR
⇒ Apply appropriate boundary conditions for pressure

Velocity Correction:

- Update velocities using pressure gradient
- Ensure mass conservation

Update Viscosity:

∑ Calculate shear rate from velocity gradients
∑ Update viscosity using Bird-Carreau model

Check Convergence:

➢ Calculate residual between iterations
➢ Exit loop if convergence criteria met

Adaptive Time Step Calculation:

➢ Calculate maximum velocity in domain
➢ Adjust time step based on CFL condition
➢ Ensure numerical stability

4. Output Results
   ❖ Periodically save flow field data to VTK files
   ❖ Record simulation statistics (residuals, velocities)
   ❖ Generate visualization data for ParaView
5. Finalization
   ✓ Write final output files
   ✓ Free allocated memory
   ✓ Report simulation performance metrics

```
/// Blood through arteries (LAD) artery analysis
// The flow is assumed to be unsteady, incompressible, laminar, Newtonian, and fully developed

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <cuda.h>
#include <cuda_runtime.h>

// Simulation parameters
#define rho 1.06e3f
#define L  0.14f
#define W  0.0035f
#define U  0.39f
#define V  0.0f
#define DX 0.0001f
#define DY 0.0001f
#define dt 1.0e-5f
#define NX ((int)(L/DX)+1)

#define NY ((int)(W/DY)+1)
#define N  (NX*NY)
#define del_P 2.8863e3f
#define mu0    0.056f
#define mu_inf 0.0035f
#define lambda 3.13f
#define n      0.6f
#define a      2.0f

// Function prototypes for kernels
__global__ void Initialize(float *u, float *v, float *p, float *visc, float *shear_rate, float p_init, int nx, int ny);
__global__ void ApplyBoundaryConditions(float *u, float *v, float *p, float p_out, int nx, int ny);
__global__ void Shear_rate(float *u, float *v, float *mu, float *shear_rate, int nx, int ny);
__global__ void Xmomentum_solver(float *u_x, float *u_y, float *p, float *mu, float *u_x_new, int nx, int ny);
__global__ void YMomentum_solver(float* u_x, float* u_y, float* p, float* v_new, float dt, float dx, float dy, float rho, float* mu, int nx, int ny);
__global__ void Divergence(float *u_x_new, float *v_new, float *divergence, int nx, int ny);
__global__ void PoissonSolverGS(float *divergence, float *p, float *p_new, int nx, int ny, float omega, float *residual);
__global__ void CorrectVelocities(float *u, float *v, float *u_star, float *v_star, float *p, int nx, int ny);
__global__ void CalculateResidual(float *u, float *u_old, float *v, float *v_old, float *residual, int nx, int ny);
__global__ void CalculateMaxVelocity(float *u, float *v, float *max_velocity, int nx, int ny);
void saveVTKFile(float *u, float *v, float *p, float *mu, float *shear_rate, int nx, int ny, int timestep);
bool SolvePressurePoisson(float *d_divergence, float *d_p, float *d_p_new, int nx, int ny, float omega, float tolerance);


//initializing the variables for the simulation
// Device arrays for staggered grid
__device__ float u[(NX+1)*NY];     // x-velocity at east/west cell faces
```

```
__device__ float v[NX*(NY+1)];     // y-velocity at north/south cell faces
__device__ float p[NX*NY];         // pressure at cell centers
__device__ float visc[NX*NY];      // viscosity at cell centers
__device__ float shear_rate[NX*NY]; // shear rate at cell centers

// Arrays for SIMPLE algorithm
__device__ float u_star[(NX+1)*NY];  // intermediate x-velocity
__device__ float v_star[NX*(NY+1)];  // intermediate y-velocity
__device__ float p_prime[NX*NY];     // pressure correction
__device__ float u_old[(NX+1)*NY];   // previous iteration x-velocity
__device__ float v_old[NX*(NY+1)];   // previous iteration y-velocity

// Host arrays
float *h_u, *h_v, *h_p, *h_visc, *h_shear_rate;

// Allocate host memory
h_u = (float*)malloc(sizeof(float) * (NX+1) * NY);
h_v = (float*)malloc(sizeof(float) * NX * (NY+1));
h_p = (float*)malloc(sizeof(float) * NX * NY);
h_visc = (float*)malloc(sizeof(float) * NX * NY);
h_shear_rate = (float*)malloc(sizeof(float) * NX * NY);

// Device pointers
float *d_u, *d_v, *d_p, *d_visc, *d_shear_rate;
float *d_u_star, *d_v_star, *d_p_prime;

// Allocate device memory
cudaMalloc((void**)&d_u, sizeof(float) * (NX+1) * NY);
cudaMalloc((void**)&d_v, sizeof(float) * NX * (NY+1));
cudaMalloc((void**)&d_p, sizeof(float) * NX * NY);
cudaMalloc((void**)&d_visc, sizeof(float) * NX * NY);
cudaMalloc((void**)&d_shear_rate, sizeof(float) * NX * NY);
cudaMalloc((void**)&d_u_star, sizeof(float) * (NX+1) * NY);
cudaMalloc((void**)&d_v_star, sizeof(float) * NX * (NY+1));
cudaMalloc((void**)&d_p_prime, sizeof(float) * NX * NY);

// Initialize host arrays
// Initialize pressure (cell centers)
for (int j = 0; j < NY; j++) {
    for (int i = 0; i < NX; i++) {
        int idx = i + j*NX;
        // Linear pressure drop from inlet to outlet
        h_p[idx] = del_P * (1.0f - (float)i/NX);
        // Initialize viscosity to zero-shear value
        h_visc[idx] = mu0;
        // Initialize shear rate to zero
        h_shear_rate[idx] = 0.0f;
    }
}

// Initialize x-velocity (east/west cell faces)
for (int j = 0; j < NY; j++) {
    for (int i = 0; i <= NX; i++) {
        int idx = i + j*(NX+1);
        h_u[idx] = 0.0f;

        // Set inlet boundary condition (west boundary, i=0)
        if (i == 0) {
            // Parabolic profile at inlet
            float y_rel = (float)j/(NY-1) - 0.5f;  // -0.5 to 0.5
```

```
            h_u[idx] = U * (1.0f - 4.0f * y_rel * y_rel);  // Parabolic profile
        }
    }
}

// Initialize y-velocity (north/south cell faces)
for (int j = 0; j <= NY; j++) {
    for (int i = 0; i < NX; i++) {
        int idx = i + j*NX;
        h_v[idx] = 0.0f;
    }
}

// Copy initial data from host to device
cudaMemcpy(d_p, h_p, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_visc, h_visc, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_shear_rate, h_shear_rate, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_u, h_u, sizeof(float)*(NX+1)*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_v, h_v, sizeof(float)*NX*(NY+1), cudaMemcpyHostToDevice);

/// Initialize flow field with initial conditions
__global__ void Initialize(float *u, float *v, float *p, float *visc, float *shear_rate,
float p_init, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Initialize pressure and viscosity at cell centers
    if (i < nx && j < ny) {
        int idx = i + j*nx;  // Cell-centered index
        p[idx] = p_init * (1.0f - (float)i/nx);  // Linear pressure gradient
        visc[idx] = mu0;  // Initialize with zero-shear viscosity
        shear_rate[idx] = 0.0f;  // Initialize shear rate to zero
    }

    // Initialize x-velocity at east/west faces (staggered grid)
    if (i <= nx && j < ny) {  // Note: i can be nx for the last face
        int idx_u = i + j*(nx+1);  // Face-centered index for u

        if (i == 0) {
            // Inlet boundary condition - uniform flow for non-pulsatile
            u[idx_u] = U;
        } else if (i == nx) {
            // Outlet boundary condition
            u[idx_u] = u[idx_u-1];  // Zero gradient
        } else {
            // Interior points
            u[idx_u] = 0.0f;  // Initialize to zero
        }
    }

    // Initialize y-velocity at north/south faces (staggered grid)
    if (i < nx && j <= ny) {  // Note: j can be ny for the last face
        int idx_v = i + j*nx;  // Face-centered index for v
        v[idx_v] = 0.0f;  // Initialize to zero

        // No-slip boundary conditions at walls (north/south)
        if (j == 0 || j == ny) {
            v[idx_v] = 0.0f;
        }
    }
```

```
}

//Boundary conditions for the flow field
// Apply boundary conditions for staggered grid
__global__ void ApplyBoundaryConditions(float *u, float *v, float *p, float p_out, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Inlet boundary (west face, x = 0)
    if (i == 0 && j < ny) {
        // u-velocity at inlet face (i=0)
        u[j*(nx+1) + i] = U;  // Uniform inlet velocity

        // v-velocity near inlet
        if (j < ny) {
            v[j*nx + i] = 0.0f;  // No y-component at inlet
        }

        // Pressure at inlet cells
        p[j*nx + i] = p[j*nx + (i+1)];  // Zero gradient
    }

    // Outlet boundary (east face, x = nx)
    if (i == nx && j < ny) {
        // u-velocity at outlet face (i=nx)
        u[j*(nx+1) + i] = u[j*(nx+1) + (i-1)];  // Zero gradient

        // Pressure at outlet cells (i=nx-1)
        if (i-1 >= 0) {
            p[j*nx + (i-1)] = p_out;  // Fixed pressure at outlet
        }
    }

    // Wall boundaries (south face, y = 0)
    if (j == 0 && i < nx) {
        // v-velocity at bottom wall (j=0)
        v[j*nx + i] = 0.0f;  // No-slip condition

        // u-velocity near bottom wall
        if (i < nx+1) {
            u[j*(nx+1) + i] = 0.0f;  // No-slip condition
        }
    }

    // Wall boundaries (north face, y = ny)
    if (j == ny && i < nx) {
        // v-velocity at top wall (j=ny)
        v[j*nx + i] = 0.0f;  // No-slip condition

        // u-velocity near top wall
        if (i < nx+1) {
            u[(j-1)*(nx+1) + i] = 0.0f;  // No-slip condition
        }
    }
}

// Calculate shear rate and update viscosity using Bird-Carreau model
__global__ void Shear_rate(float *u, float *v, float *mu, float *shear_rate, int nx, int ny) {
```

```
225     int i = blockIdx.x * blockDim.x + threadIdx.x;
226     int j = blockIdx.y * blockDim.y + threadIdx.y;
227 
228     if (i > 0 && i < nx-1 && j > 0 && j < ny-1) {
229         // For staggered grid, u is at (i+1/2,j) and v is at (i,j+1/2)
230         // Calculate velocity gradients at cell center (i,j)
231         float du_dx = (u[j*(nx+1) + (i+1)] - u[j*(nx+1) + i]) / DX;
232         float dv_dy = (v[(j+1)*nx + i] - v[j*nx + i]) / DY;
233 
234         // Cross derivatives require averaging
235         float du_dy = (u[(j+1)*(nx+1) + i] + u[(j+1)*(nx+1) + (i+1)] -u[j*(nx+1) + i] -
    u[j*(nx+1) + (i+1)]) / (2.0f * DY);
236         float dv_dx = (v[j*nx + (i+1)] + v[(j+1)*nx + (i+1)] -v[j*nx + i] - v[(j+1)*nx +
    i]) / (2.0f * DX);
237 
238         // Calculate shear rate
239         float sr = sqrtf(2.0f * (du_dx*du_dx + dv_dy*dv_dy) + (du_dy + dv_dx)*(du_dy +
    dv_dx));
240 
241         // Store at cell center
242         int idx = j*nx + i;
243         shear_rate[idx] = sr;
244 
245         // Update viscosity using Bird-Carreau model
246         mu[idx] = mu_inf + (mu0 - mu_inf) * powf(1.0f + powf(lambda * sr, a), (n-1.0f)/a);
247     } else if (i < nx && j < ny) {
248         // Set boundary values to zero or a safe default
249         int idx = j*nx + i;
250         shear_rate[idx] = 0.0f;
251         mu[idx] = mu0;
252     }
253 }
254 
255 //////////*************************SIMPLE ALGORITHM
    STARTED*********************************//////////////////
256 
257 //solver for x momentum equation
258 //Xmomentum equation solver
259 __global__ void Xmomentum_solver(float *u_x, float *u_y, float *p, float *mu, float
    *u_x_new, int nx, int ny) {
260     int i = blockIdx.x * blockDim.x + threadIdx.x;
261     int j = blockIdx.y * blockDim.y + threadIdx.y;
262 
263     if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
264         // For staggered grid, u_x is stored at (i+1/2, j)
265         // Proper indexing for u_x: j*(nx+1) + i
266         int idx_u = j*(nx+1) + i;
267 
268         // 1. Calculate convection term using QUICK scheme
269         // Calculate mass fluxes through cell faces
270         float Fe = 0.5f * (u_x[j*(nx+1) + i] + u_x[j*(nx+1) + (i+1)]) * DY;
271         float Fw = 0.5f * (u_x[j*(nx+1) + (i-1)] + u_x[j*(nx+1) + i]) * DY;
272 
273         // Interpolate v-velocities to u-cell faces
274         float vn = 0.25f * (u_y[(j+1)*nx + i] + u_y[(j+1)*nx + (i-1)] +u_y[j*nx + i] +
    u_y[j*nx + (i-1)]);
275         float vs = 0.25f * (u_y[j*nx + i] + u_y[j*nx + (i-1)] +u_y[(j-1)*nx + i] +
    u_y[(j-1)*nx + (i-1)]);
276 
277         float Fn = vn * DX;
```

```
278         float Fs = vs * DX;
279 
280         // QUICK scheme for face values
281         float phi_e, phi_w, phi_n, phi_s;
282 
283         // East face
284         if (Fe >= 0) {
285             phi_e = (3.0f/8.0f) * u_x[j*(nx+1) + (i+1)] +(6.0f/8.0f) * u_x[j*(nx+1) + i] -
    (1.0f/8.0f) * u_x[j*(nx+1) + (i-1)];
286         } else {
287             phi_e = (3.0f/8.0f) * u_x[j*(nx+1) + i] +(6.0f/8.0f) * u_x[j*(nx+1) + (i+1)] -
    (1.0f/8.0f) * u_x[j*(nx+1) + (i+2)];
288         }
289 
290         // West face
291         if (Fw >= 0) {
292             phi_w = (3.0f/8.0f) * u_x[j*(nx+1) + i] + (6.0f/8.0f) * u_x[j*(nx+1) + (i-1)]
    -(1.0f/8.0f) * u_x[j*(nx+1) + (i-2)];
293         } else {
294             phi_w = (3.0f/8.0f) * u_x[j*(nx+1) + (i-1)] + (6.0f/8.0f) * u_x[j*(nx+1) + i]
    - (1.0f/8.0f) * u_x[j*(nx+1) + (i+1)];
295         }
296 
297         // North face
298         if (Fn >= 0) {
299             phi_n = (3.0f/8.0f) * u_x[(j+1)*(nx+1) + i] +(6.0f/8.0f) * u_x[j*(nx+1) + i] -
    (1.0f/8.0f) * u_x[(j-1)*(nx+1) + i];
300         } else {
301             phi_n = (3.0f/8.0f) * u_x[j*(nx+1) + i] +(6.0f/8.0f) * u_x[(j+1)*(nx+1) + i] -
    (1.0f/8.0f) * u_x[(j+2)*(nx+1) + i];
302         }
303 
304         // South face
305         if (Fs >= 0) {
306             phi_s = (3.0f/8.0f) * u_x[j*(nx+1) + i] + (6.0f/8.0f) * u_x[(j-1)*(nx+1) + i]
    - (1.0f/8.0f) * u_x[(j-2)*(nx+1) + i];
307         } else {
308             phi_s = (3.0f/8.0f) * u_x[(j-1)*(nx+1) + i] + (6.0f/8.0f) * u_x[j*(nx+1) + i]
    -(1.0f/8.0f) * u_x[(j+1)*(nx+1) + i];
309         }
310 
311         float convection_term = -(Fe * phi_e - Fw * phi_w + Fn * phi_n - Fs * phi_s) / (DX
    * DY);
312 
313         // 2. Calculate diffusion term using central difference
314         // Interpolate viscosity to u-cell faces
315         float mu_e = 0.5f * (mu[j*nx + i] + mu[j*nx + (i+1)]);
316         float mu_w = 0.5f * (mu[j*nx + (i-1)] + mu[j*nx + i]);
317         float mu_n = 0.5f * (mu[j*nx + i] + mu[(j+1)*nx + i]);
318         float mu_s = 0.5f * (mu[(j-1)*nx + i] + mu[j*nx + i]);
319 
320         float diffusion_term = (mu_e * (u_x[j*(nx+1) + (i+1)] - u_x[j*(nx+1) + i]) / DX -
    mu_w * (u_x[j*(nx+1) + i] - u_x[j*(nx+1) + (i-1)]) / DX) / DX + (mu_n * (u_x[(j+1)*(nx+1)
    + i] - u_x[j*(nx+1) + i]) / DY -mu_s * (u_x[j*(nx+1) + i] - u_x[(j-1)*(nx+1) + i]) / DY) /
    DY;
321 
322         // 3. Calculate pressure gradient (staggered grid - pressure at cell centers)
323         // For u_x at (i+1/2,j), we need pressure gradient between p[i,j] and p[i+1,j]
324         float pressure_gradient_term = (p[j*nx + i] - p[j*nx + (i-1)]) / DX;
325 
```

```
326         // 4. Update u_x using the momentum equation
327         u_x_new[j*(nx+1) + i] = u_x[j*(nx+1) + i] +dt * (-convection_term +
    diffusion_term/rho - pressure_gradient_term/rho);
328     }
329 }
330
331 //solver for y momentum equation
332 //Ymomentum equation solver
333 //solver for y momentum equation
334 //Ymomentum equation solver
335 __global__ void YMomentum_solver(float* u_x, float* u_y, float* p, float* v_new, float dt,
    float dx, float dy, float rho, float* mu, int nx, int ny) {
336     int i = blockIdx.x * blockDim.x + threadIdx.x;
337     int j = blockIdx.y * blockDim.y + threadIdx.y;
338     if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {  // Fixed upper bound check for j
339         // For staggered grid, v is at (i,j+1/2)
340         int idx = i + j*nx;  // V_IDX(i, j, nx)
341
342         float dx_e = dx, dx_w = dx, dy_n = dy, dy_s = dy;
343
344         // Interpolate u-velocities to v-cell faces (east and west)
345         float ue = 0.25f * (u_x[i + j*(nx+1)] + u_x[(i+1) + j*(nx+1)] +u_x[i +
    (j-1)*(nx+1)] + u_x[(i+1) + (j-1)*(nx+1)]);
346         float Fe = ue * dy_n;
347         float uw = 0.25f * (u_x[(i-1) + j*(nx+1)] + u_x[i + j*(nx+1)] + u_x[(i-1) +
    (j-1)*(nx+1)] + u_x[i + (j-1)*(nx+1)]);
348         float Fw = uw * dy_s;
349
350         // Interpolate v-velocities to v-cell faces (north and south)
351         float vn = 0.5f * (u_y[i + j*nx] + u_y[i + (j+1)*nx]);
352         float Fn = vn * dx_e;
353         float vs = 0.5f * (u_y[i + (j-1)*nx] + u_y[i + j*nx]);
354         float Fs = vs * dx_w;
355
356         // QUICK scheme for face values with flow direction check
357         float phi_e, phi_w, phi_n, phi_s;
358
359         // East face
360         if (Fe >= 0) {
361             phi_e = (3.0f/8.0f) * u_y[(i+1) + j*nx] + (6.0f/8.0f) * u_y[i + j*nx] -
    (1.0f/8.0f) * u_y[(i-1) + j*nx];
362
363
364         } else {
365             phi_e = (3.0f/8.0f) * u_y[i + j*nx] + (6.0f/8.0f) * u_y[(i+1) + j*nx] - (1.0f/
    8.0f) * u_y[(i+2) + j*nx];
366         }
367
368         // West face
369         if (Fw >= 0) {
370             phi_w = (3.0f/8.0f) * u_y[i + j*nx] + (6.0f/8.0f) * u_y[(i-1) + j*nx] - (1.0f/
    8.0f) * u_y[(i-2) + j*nx];
371
372         } else {
373             phi_w = (3.0f/8.0f) * u_y[(i-1) + j*nx] + (6.0f/8.0f) * u_y[i + j*nx] - (1.0f/
    8.0f) * u_y[(i+1) + j*nx];
374         }
375
376         // North face
377         if (Fn >= 0) {
```

```
378             phi_n = (3.0f/8.0f) * u_y[i + (j+1)*nx] + (6.0f/8.0f) * u_y[i + j*nx] - (1.0f/
    8.0f) * u_y[i + (j-1)*nx];
379         } else {
380             phi_n = (3.0f/8.0f) * u_y[i + j*nx] +(6.0f/8.0f) * u_y[i + (j+1)*nx] -(1.0f/
    8.0f) * u_y[i + (j+2)*nx];
381         }
382 
383         // South face
384         if (Fs >= 0) {
385             phi_s = (3.0f/8.0f) * u_y[i + j*nx] +(6.0f/8.0f) * u_y[i + (j-1)*nx] -(1.0f/
    8.0f) * u_y[i + (j-2)*nx];
386         } else {
387             phi_s = (3.0f/8.0f) * u_y[i + (j-1)*nx] +(6.0f/8.0f) * u_y[i + j*nx] - (1.0f/
    8.0f) * u_y[i + (j+1)*nx];
388         }
389 
390         // Viscosity interpolation - corrected indexing for staggered grid
391         float mu_e = 0.5f * (mu[i + j*nx] + mu[(i+1) + j*nx]);
392         float mu_w = 0.5f * (mu[(i-1) + j*nx] + mu[i + j*nx]);
393         float mu_n = 0.5f * (mu[i + j*nx] + mu[i + (j+1)*nx]);
394         float mu_s = 0.5f * (mu[i + (j-1)*nx] + mu[i + j*nx]);
395 
396         float De = mu_e * dy_n / dx_e;
397         float Dw = mu_w * dy_s / dx_w;
398         float Dn = mu_n * dx_e / dy_n;
399         float Ds = mu_s * dx_w / dy_s;
400 
401         // Pressure gradient - corrected for v-velocity at (i,j+1/2)
402         float dp_dy = (p[i + j*nx] - p[i + (j-1)*nx]) / dy;
403 
404         float convection = Fe * phi_e - Fw * phi_w + Fn * phi_n - Fs * phi_s;
405         float diffusion = De * (u_y[(i+1) + j*nx] - u_y[i + j*nx])- Dw * (u_y[i + j*nx] -
    u_y[(i-1) + j*nx])+ Dn * (u_y[i + (j+1)*nx] - u_y[i + j*nx])- Ds * (u_y[i + j*nx] - u_y[i
    + (j-1)*nx]);
406 
407         v_new[idx] = u_y[idx] + dt * (-convection/(dx*dy) + diffusion/(dx*dy) - dp_dy/
    rho);
408     }
409 }
410 
411 
412 // SOLVER FOR PRESSURE CORRECTION STEP
413 // Pressure correction solver
414 //COMPUTING THE DIVERGENCE FOR BOTH U_NEW AND V_NEW
415 
416 __global__ void Divergence(float *u_x_new, float *v_new, float *divergence, int nx, int
    ny) {
417     int i = blockIdx.x * blockDim.x + threadIdx.x;
418     int j = blockIdx.y * blockDim.y + threadIdx.y;
419 
420     if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
421         // For staggered grid:
422         // u_x is stored at (i+1/2,j) - east/west cell faces
423         // v is stored at (i,j+1/2) - north/south cell faces
424 
425         // Calculate divergence using central difference
426         // u_x at east face (i+1/2,j) minus u_x at west face (i-1/2,j)
427         float du_dx = (u_x_new[j*(nx+1) + (i+1)] - u_x_new[j*(nx+1) + i]) / DX;
428 
429         // v at north face (i,j+1/2) minus v at south face (i,j-1/2)
```

```
430         float dv_dy = (v_new[(j+1)*nx + i] - v_new[j*nx + i]) / DY;
431 
432         // Store divergence at cell center (i,j)
433         divergence[j*nx + i] = du_dx + dv_dy;
434     }
435 }
436 
437 // Pressure correction solver
438 // Pressure Poisson equation solver using Gauss-Seidel with SOR
439 __global__ void PoissonSolverGS(float *divergence, float *p, float *p_new, int nx, int ny, float omega, float *residual) {
440     int i = blockIdx.x * blockDim.x + threadIdx.x;
441     int j = blockIdx.y * blockDim.y + threadIdx.y;
442 
443     if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
444         // Proper indexing for staggered grid (cell centers)
445         int idx = j*nx + i;  // Row-major order
446 
447         float source = -rho * divergence[idx] / dt;
448         float p_old = p[idx];
449 
450         // Gauss-Seidel iteration for pressure correction
451         float p_gs = 0.25f * (
452             p_new[j*nx + (i+1)] + p_new[j*nx + (i-1)] +
453             p_new[(j+1)*nx + i] + p_new[(j-1)*nx + i] -
454             DX * DY * source
455         );
456 
457         // Apply SOR (Successive Over-Relaxation)
458         p_new[idx] = (1.0f - omega) * p_old + omega * p_gs;
459 
460         // Calculate local residual for convergence check
461         // Using atomicAdd can cause performance issues due to thread contention
462         // Better to compute local residuals and reduce them later
463         atomicAdd(residual, fabsf(p_new[idx] - p_old));
464     }
465 }
466 
467 // CONVERGENCE CHECK FOR Host-side wrapper for Gauss-Seidel iteration with convergence check
468 // Host-side wrapper for Gauss-Seidel iteration with convergence check
469 bool SolvePressurePoisson(float *d_divergence, float *d_p, float *d_p_new, int nx, int ny, float omega, float tolerance) {
470     dim3 block(16, 16);
471     dim3 grid((nx + block.x - 1) / block.x, (ny + block.y - 1) / block.y);
472 
473     float *d_residual;
474     cudaMalloc((void**)&d_residual, sizeof(float));
475     float h_residual;
476 
477     int max_iter = 1000;  // Maximum iterations
478 
479     for (int iter = 0; iter < max_iter; ++iter) {
480         // Reset residual
481         cudaMemset(d_residual, 0, sizeof(float));
482 
483         // Copy current pressure to p_new as initial guess
484         cudaMemcpy(d_p_new, d_p, nx * ny * sizeof(float), cudaMemcpyDeviceToDevice);
485 
486         // Perform one Gauss-Seidel iteration
```

```
        PoissonSolverGS<<<grid, block>>>(d_divergence, d_p, d_p_new, nx, ny, omega, d_residual);
        cudaDeviceSynchronize();

        // Check convergence
        cudaMemcpy(&h_residual, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
        h_residual /= (nx * ny);  // Normalize by grid size

        if (h_residual < tolerance) {
            cudaFree(d_residual);
            return true;  // Converged
        }

        // Update pressure for next iteration
        cudaMemcpy(d_p, d_p_new, nx * ny * sizeof(float), cudaMemcpyDeviceToDevice);
    }

    cudaFree(d_residual);
    return false;  // Did not converge within max_iter
}


// Correct velocities using pressure correction based on SIMPLE algorithm using staggered grid
//with time step implementation
__global__ void CorrectVelocities(float *u, float *v, float *u_star, float *v_star, float *p, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Correct u-velocity (staggered at i+1/2, j)
    if (i >= 1 && i < nx && j >= 1 && j < ny-1) {
        // d-coefficient from momentum equation (simplified)
        float d_u = dt / rho;

        // Proper indexing for staggered grid
        int idx_u = j*(nx+1) + i;  // u at (i+1/2,j)

        // Pressure gradient between adjacent cell centers
        float dp_dx = (p[j*nx + i] - p[j*nx + (i-1)]) / DX;

        // Correct u-velocity using pressure gradient
        u[idx_u] = u_star[idx_u] - d_u * dp_dx;
    }

    // Correct v-velocity (staggered at i, j+1/2)
    if (i >= 1 && i < nx-1 && j >= 1 && j < ny) {
        // d-coefficient from momentum equation (simplified)
        float d_v = dt / rho;

        // Proper indexing for staggered grid
        int idx_v = j*nx + i;  // v at (i,j+1/2)

        // Pressure gradient between adjacent cell centers
        float dp_dy = (p[j*nx + i] - p[(j-1)*nx + i]) / DY;

        // Correct v-velocity using pressure gradient
        v[idx_v] = v_star[idx_v] - d_v * dp_dy;
    }
}
```

```
544
545 // Main temporal simulation loop
546 __host__ void runSimulation(float *d_u, float *d_v, float *d_p, float *d_visc, float
    *d_shear_rate, float *d_u_star, float *d_v_star, float *d_p_prime, float *d_divergence) {
547     // Simulation parameters
548     float physicalTime = 5.0f;        // Total physical time to simulate (seconds)
549     float currentTime = 0.0f;         // Current physical time
550     int timeStep = 0;                 // Current time step
551     float cfl = 1.0f;                 // Target CFL number
552     float maxResidual = 1.0f;         // Initial residual
553     float convergenceTolerance = 1e-5f; // Convergence criteria
554     int maxIterations = 10000;         // Maximum SIMPLE iterations per time step
555     int outputInterval = 100;         // Output every N time steps
556
557     // Ask user for simulation parameters
558     printf("Enter total physical time (default: 5.0s): ");
559     scanf("%f", &physicalTime);
560     printf("Enter output interval in time steps (default: 100): ");
561     scanf("%d", &outputInterval);
562
563     // Calculate adaptive time step based on CFL condition
564     float maxVelocity = U;  // Initial estimate
565     float adaptiveDt = cfl * DX / maxVelocity;
566     printf("Initial time step: %e seconds\n", adaptiveDt);
567
568     // Allocate host memory for monitoring
569     float *h_u = (float*)malloc(sizeof(float) * (NX+1) * NY);
570     float *h_v = (float*)malloc(sizeof(float) * NX * (NY+1));
571     float *h_p = (float*)malloc(sizeof(float) * NX * NY);
572     float *h_residual = (float*)malloc(sizeof(float));
573
574     // Execution configuration
575     dim3 block(16, 16);
576     dim3 grid((NX + block.x - 1) / block.x, (NY + block.y - 1) / block.y);
577
578     // Create output file
579     FILE *outfile = fopen("blood_flow_results.csv", "w");
580     fprintf(outfile, "Time,TimeStep,Iterations,MaxResidual,MaxVelocity,MaxPressure\n");
581
582     // Time advancement loop
583     while (currentTime < physicalTime) {
584         timeStep++;
585
586         // Store previous time step values for temporal discretization
587         cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToDevice);
588         cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToDevice);
589
590         // SIMPLE algorithm iterations
591         int iterations = 0;
592         maxResidual = 1.0f;
593
594         while (iterations < maxIterations && maxResidual > convergenceTolerance) {
595             // Step 1: Momentum prediction
596             Xmomentum_solver<<<grid, block>>>(d_u, d_v, d_p, d_visc, d_u_star, NX, NY);
597            YMomentum_solver<<<grid, block>>>(d_u, d_v, d_p, d_v_star, adaptiveDt, DX, DY,
    rho, d_visc, NX, NY);
598
599             // Step 2: Calculate divergence
600             Divergence<<<grid, block>>>(d_u_star, d_v_star, d_divergence, NX, NY);
601
```

```
602             // Step 3: Solve pressure correction equation
603             float omega = 1.5f;  // SOR relaxation parameter
604             bool converged = SolvePressurePoisson(d_divergence, d_p, d_p_prime, NX, NY,
    omega, convergenceTolerance/10.0f);
605 
606             // Step 4: Correct velocities and pressure
607             CorrectVelocities<<<grid, block>>>(d_u, d_v, d_u_star, d_v_star, d_p, NX, NY);
608 
609             // Step 5: Update viscosity using Bird-Carreau model
610             Shear_rate<<<grid, block>>>(d_u, d_v, d_visc, d_shear_rate, NX, NY);
611 
612             // Calculate residual for convergence check
613             float *d_residual;
614             cudaMalloc((void**)&d_residual, sizeof(float));
615             cudaMemset(d_residual, 0, sizeof(float));
616 
617             // Calculate velocity residual (change between iterations)
618             CalculateResidual<<<grid, block>>>(d_u, d_u_old, d_v, d_v_old, d_residual, NX,
    NY);
619             cudaMemcpy(h_residual, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
620             maxResidual = *h_residual;
621 
622             cudaFree(d_residual);
623             iterations++;
624 
625             // Store current values for next iteration
626             cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY,
    cudaMemcpyDeviceToDevice);
627             cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1),
    cudaMemcpyDeviceToDevice);
628         }
629 
630         // Update current time
631         currentTime += adaptiveDt;
632 
633         // Calculate maximum velocity for adaptive time stepping
634         float *d_max_velocity;
635         cudaMalloc((void**)&d_max_velocity, sizeof(float));
636         cudaMemset(d_max_velocity, 0, sizeof(float));
637 
638         CalculateMaxVelocity<<<grid, block>>>(d_u, d_v, d_max_velocity, NX, NY);
639         float h_max_velocity;
640         cudaMemcpy(&h_max_velocity, d_max_velocity, sizeof(float),
    cudaMemcpyDeviceToHost);
641         cudaFree(d_max_velocity);
642 
643         // Update time step based on CFL condition
644         if (h_max_velocity > 0.01f) {  // Avoid division by very small values
645             adaptiveDt = cfl * DX / h_max_velocity;
646         }
647 
648         // Limit time step growth for stability
649         adaptiveDt = fminf(adaptiveDt, 1.5f * adaptiveDt);
650 
651         // Adjust final time step to exactly reach physicalTime
652         if (currentTime + adaptiveDt > physicalTime) {
653             adaptiveDt = physicalTime - currentTime;
654         }
655 
656         // Output results at specified intervals
```

```
657         if (timeStep % outputInterval == 0 || currentTime >= physicalTime) {
658             printf("Time: %.4f s, Step: %d, Iterations: %d, Residual: %.6e, Max Velocity:
    %.4f m/s\n",
659                    currentTime, timeStep, iterations, maxResidual, h_max_velocity);
660 
661             // Copy data back to host for output
662             cudaMemcpy(h_u, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToHost);
663             cudaMemcpy(h_v, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToHost);
664             cudaMemcpy(h_p, d_p, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);
665 
666             // Calculate maximum pressure
667             float maxPressure = 0.0f;
668             for (int i = 0; i < NX*NY; i++) {
669                 if (fabs(h_p[i]) > maxPressure) maxPressure = fabs(h_p[i]);
670             }
671 
672             // Write to output file
673             fprintf(outfile, "%.6f,%d,%d,%.6e,%.6f,%.6f\n",
674                     currentTime, timeStep, iterations, maxResidual, h_max_velocity,
    maxPressure);
675 
676             // Write field data to VTK file for visualization
677             char filename[256];
678             sprintf(filename, "blood_flow_t%.4f.vtk", currentTime);
679             writeVTKFile(filename, h_u, h_v, h_p, NX, NY, DX, DY);
680         }
681     }
682 
683     // Close output file
684     fclose(outfile);
685 
686     // Free host memory
687     free(h_u);
688     free(h_v);
689     free(h_p);
690     free(h_visc);
691     free(h_shear_rate);
692 
693     printf("Simulation completed: %.4f seconds simulated in %d time steps\n", currentTime,
    timeStep);
694 }
695 
696 // Function to save data in VTK format for ParaView visualization
697 // Function to save data in VTK format for ParaView visualization
698 void saveVTKFile(float *u, float *v, float *p, float *mu, float *shear_rate, int nx, int
    ny, int timestep) {
699     char filename[256];
700     sprintf(filename, "blood_flow_%04d.vtk", timestep);
701     FILE *fp = fopen(filename, "w");
702     if (fp == NULL) {
703         printf("Error opening file for VTK output\n");
704         return;
705     }
706 
707     // VTK header
708     fprintf(fp, "# vtk DataFile Version 3.0\n");
709     fprintf(fp, "LAD Artery Blood Flow Simulation\n");
710     fprintf(fp, "ASCII\n");
711     fprintf(fp, "DATASET STRUCTURED_POINTS\n");
712     fprintf(fp, "DIMENSIONS %d %d 1\n", nx, ny);
```

```
    fprintf(fp, "ORIGIN 0 0 0\n");
    fprintf(fp, "SPACING %f %f 1.0\n", DX, DY);
    fprintf(fp, "POINT_DATA %d\n", nx * ny);

    fprintf(fp, "SCALARS u_velocity float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", u[i*ny + j]);
    fprintf(fp, "SCALARS v_velocity float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", v[i*ny + j]);
    fprintf(fp, "SCALARS pressure float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", p[i*ny + j]);
    fprintf(fp, "SCALARS viscosity float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", mu[i*ny + j]);
    fprintf(fp, "SCALARS shear_rate float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", shear_rate[i*ny + j]);
    fprintf(fp, "VECTORS velocity float\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f %f 0.0\n", u[i*ny + j], v[i*ny + j]);
    fclose(fp);
    printf("Saved VTK file: %s\n", filename);
}

int main() {
    // Host arrays
    float *h_u, *h_v, *h_p, *h_visc, *h_shear_rate;
    // Device arrays
    float *d_u, *d_v, *d_p, *d_visc, *d_shear_rate;
    float *d_u_star, *d_v_star, *d_p_prime, *d_divergence;
    float *d_u_old, *d_v_old;

    // Allocate host memory
    h_u = (float*)malloc(sizeof(float) * (NX+1) * NY);
    h_v = (float*)malloc(sizeof(float) * NX * (NY+1));
    h_p = (float*)malloc(sizeof(float) * NX * NY);
    h_visc = (float*)malloc(sizeof(float) * NX * NY);
    h_shear_rate = (float*)malloc(sizeof(float) * NX * NY);

    // Allocate device memory
    cudaMalloc((void**)&d_u, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v, sizeof(float) * NX * (NY+1));
    cudaMalloc((void**)&d_p, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_visc, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_shear_rate, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_u_star, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v_star, sizeof(float) * NX * (NY+1));
    cudaMalloc((void**)&d_p_prime, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_divergence, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_u_old, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v_old, sizeof(float) * NX * (NY+1));

```

```
    // Initialize host arrays
    for (int j = 0; j < NY; j++) {
        for (int i = 0; i < NX; i++) {
            int idx = i + j*NX;
            h_p[idx] = del_P * (1.0f - (float)i/NX);
            h_visc[idx] = mu0;
            h_shear_rate[idx] = 0.0f;
        }
    }
    for (int j = 0; j < NY; j++) {
        for (int i = 0; i <= NX; i++) {
            int idx = i + j*(NX+1);
            h_u[idx] = (i == 0) ? U : 0.0f;
        }
    }
    for (int j = 0; j <= NY; j++) {
        for (int i = 0; i < NX; i++) {
            int idx = i + j*NX;
            h_v[idx] = 0.0f;
        }
    }

    // Copy initial data from host to device
    cudaMemcpy(d_p, h_p, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_visc, h_visc, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_shear_rate, h_shear_rate, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_u, h_u, sizeof(float)*(NX+1)*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_v, h_v, sizeof(float)*NX*(NY+1), cudaMemcpyHostToDevice);

    // Set execution configuration
    dim3 blockSize(16, 16);
    dim3 gridSize((NX + blockSize.x - 1) / blockSize.x, (NY + blockSize.y - 1) /
blockSize.y);

    // Simulation parameters
    float physicalTime = 5.0f;
    float currentTime = 0.0f;
    int timeStep = 0;
    float cfl = 0.5f;
    float maxResidual = 1.0f;
    float convergenceTolerance = 1e-5f;
    int maxIterations = 1000;
    int outputInterval = 100;

    printf("Enter total physical time (default: 5.0s): ");
    scanf("%f", &physicalTime);
    printf("Enter output interval in time steps (default: 100): ");
    scanf("%d", &outputInterval);

    float maxVelocity = U;
    float adaptiveDt = cfl * DX / maxVelocity;
    printf("Initial time step: %e seconds\n", adaptiveDt);

    FILE *outfile = fopen("blood_flow_results.csv", "w");
    fprintf(outfile, "Time,TimeStep,Iterations,MaxResidual,MaxVelocity,MaxPressure\n");

    while (currentTime < physicalTime) {
        timeStep++;
        cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToDevice);
        cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToDevice);
```

```
832
833         int iterations = 0;
834         maxResidual = 1.0f;
835
836         while (iterations < maxIterations && maxResidual > convergenceTolerance) {
837             ApplyBoundaryConditions<<<gridSize, blockSize>>>(d_u, d_v, d_p, 0.0f, NX, NY);
838             Xmomentum_solver<<<gridSize, blockSize>>>(d_u, d_v, d_p, d_visc, d_u_star, NX,
    NY);
839             YMomentum_solver<<<gridSize, blockSize>>>(d_u, d_v, d_p, d_v_star, adaptiveDt,
    DX, DY, rho, d_visc, NX, NY);
840             Divergence<<<gridSize, blockSize>>>(d_u_star, d_v_star, d_divergence, NX, NY);
841
842             float omega = 1.5f;
843             bool converged = SolvePressurePoisson(d_divergence, d_p, d_p_prime, NX, NY,
    omega, convergenceTolerance/10.0f);
844
845             CorrectVelocities<<<gridSize, blockSize>>>(d_u, d_v, d_u_star, d_v_star, d_p,
    NX, NY);
846             Shear_rate<<<gridSize, blockSize>>>(d_u, d_v, d_visc, d_shear_rate, NX, NY);
847
848             float *d_residual;
849             cudaMalloc((void**)&d_residual, sizeof(float));
850             cudaMemset(d_residual, 0, sizeof(float));
851             CalculateResidual<<<gridSize, blockSize>>>(d_u, d_u_old, d_v, d_v_old,
    d_residual, NX, NY);
852             cudaMemcpy(&maxResidual, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
853             cudaFree(d_residual);
854
855             iterations++;
856             cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY,
    cudaMemcpyDeviceToDevice);
857             cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1),
    cudaMemcpyDeviceToDevice);
858         }
859
860         currentTime += adaptiveDt;
861
862         float *d_max_velocity;
863         cudaMalloc((void**)&d_max_velocity, sizeof(float));
864         cudaMemset(d_max_velocity, 0, sizeof(float));
865         CalculateMaxVelocity<<<gridSize, blockSize>>>(d_u, d_v, d_max_velocity, NX, NY);
866         float h_max_velocity;
867         cudaMemcpy(&h_max_velocity, d_max_velocity, sizeof(float),
    cudaMemcpyDeviceToHost);
868         cudaFree(d_max_velocity);
869
870         if (h_max_velocity > 0.01f) {
871             adaptiveDt = cfl * DX / h_max_velocity;
872         }
873         adaptiveDt = fminf(adaptiveDt, 1.5f * adaptiveDt);
874         if (currentTime + adaptiveDt > physicalTime) {
875             adaptiveDt = physicalTime - currentTime;
876         }
877
878         if (timeStep % outputInterval == 0 || currentTime >= physicalTime) {
879             printf("Time: %.4f s, Step: %d, Iterations: %d, Residual: %.6e, Max Velocity:
    %.4f m/s\n",
880                    currentTime, timeStep, iterations, maxResidual, h_max_velocity);
881
882             cudaMemcpy(h_u, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToHost);
```

```
883             cudaMemcpy(h_v, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToHost);
884             cudaMemcpy(h_p, d_p, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);
885 
886             float maxPressure = 0.0f;
887             for (int i = 0; i < NX*NY; i++) {
888                 if (fabs(h_p[i]) > maxPressure) maxPressure = fabs(h_p[i]);
889             }
890             fprintf(outfile, "%.6f,%d,%d,%.6e,%.6f,%.6f\n",
891                     currentTime, timeStep, iterations, maxResidual, h_max_velocity,
    maxPressure);
892 
893             saveVTKFile(h_u, h_v, h_p, h_visc, h_shear_rate, NX, NY, timeStep);
894         }
895     }
896 
897     fclose(outfile);
898     free(h_u); free(h_v); free(h_p); free(h_visc); free(h_shear_rate);
899     cudaFree(d_u); cudaFree(d_v); cudaFree(d_p); cudaFree(d_visc); cudaFree(d_shear_rate);
900     cudaFree(d_u_star); cudaFree(d_v_star); cudaFree(d_p_prime); cudaFree(d_divergence);
901     cudaFree(d_u_old); cudaFree(d_v_old);
902 
903     printf("Simulation completed.\n");
904     return 0;
905 }
```